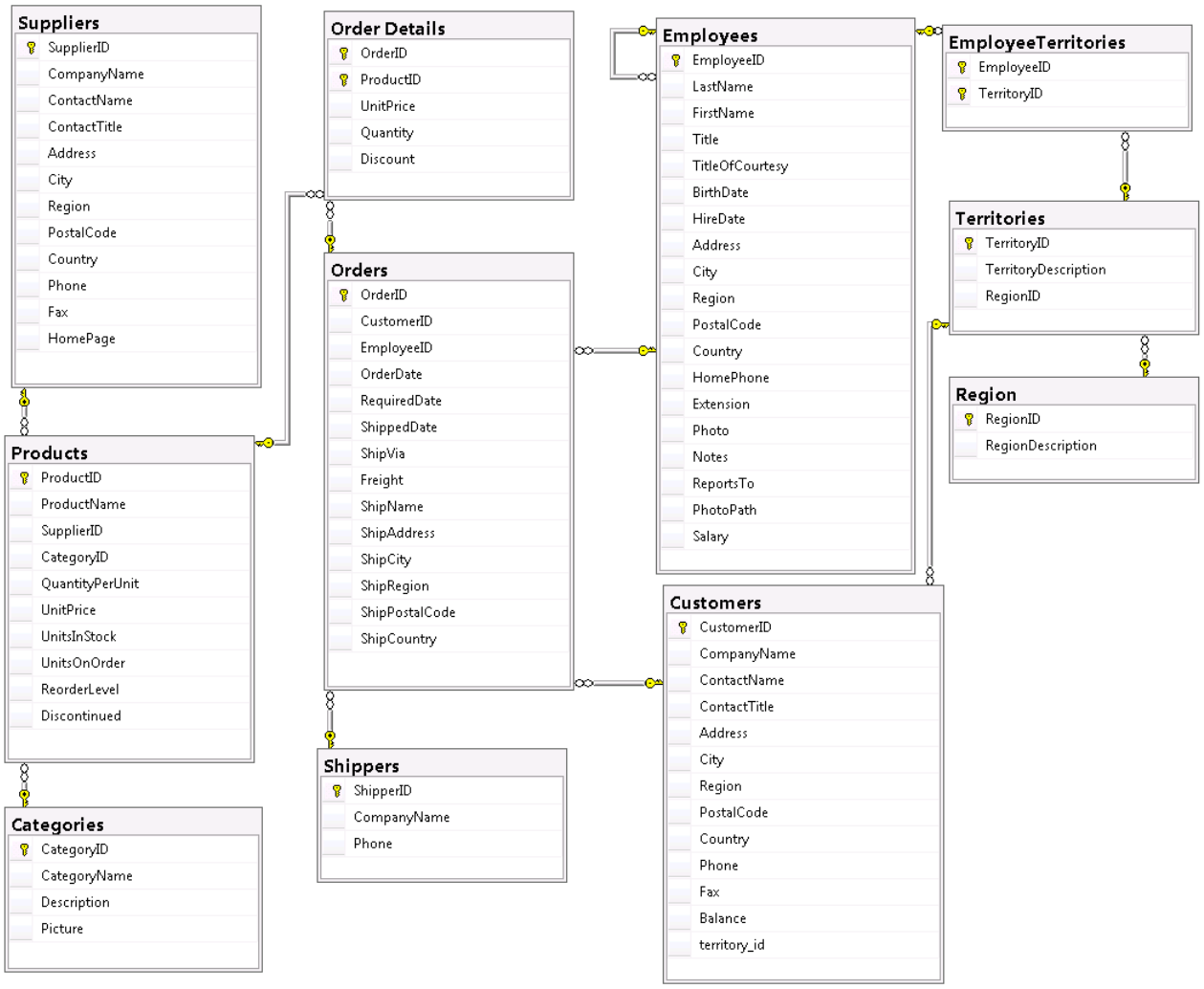
Advanced database management systems
Topic 1: Loose coupling based on triggers and jobs

Introduction

Welcome. In some of the demos in this course, we'll use the Northwind sample relational database and the SQL server 2022 technology from Microsoft. The Northwind database was designed to support a small company trading with consumables. It includes an inventory and tables for the administration of the orders. The table and file names should be self-explanatory.



Some useful resources:

- You can download the database dump from: https://github.com/microsoft/sql-server-samples/tree/master/samples/databases/northwind-pubs
- The SQL server is free to download for educational purposes from: https://www.microsoft.com/en-us/sql-server/sql-server-downloads
- You can manage the SQL Server instance with the SQL Server Management Studio (SSMS) free tool, available from: https://learn.microsoft.com/en-us/ssms/download-sql-server-management-studio-ssms

- There are several tutorials on the use of SSMS, including https://learn.microsoft.com/en-us/ssms/quickstarts/ssms-connect-query-sql-server

Problem scenario of Topic 1

The new orders are stored in the Orders and Orderitems tables by a third party management and trading application that has no open API, or for any other reason refuses to generate service level events. Therefore, in the current order processing workflow at the Northwind Traders Ltd Co. the trading department communicates with the Shipping and Logistics (SL) division via email (or any other manual messaging system) about the new or changed orders. Our company is responsible for IT support in SL management. The head of SL division prepares the detailed daily work plans every morning for the various units according to the emails received from the trading department. For this, she uses our software tool. Both the trading and the SL use the same Northwind SQL Server database.

We are asked to relieve the trading and SL staff from manually writing emails and manually entering data from emails into another application by automating the order processing workflow as much as possible.

Solution

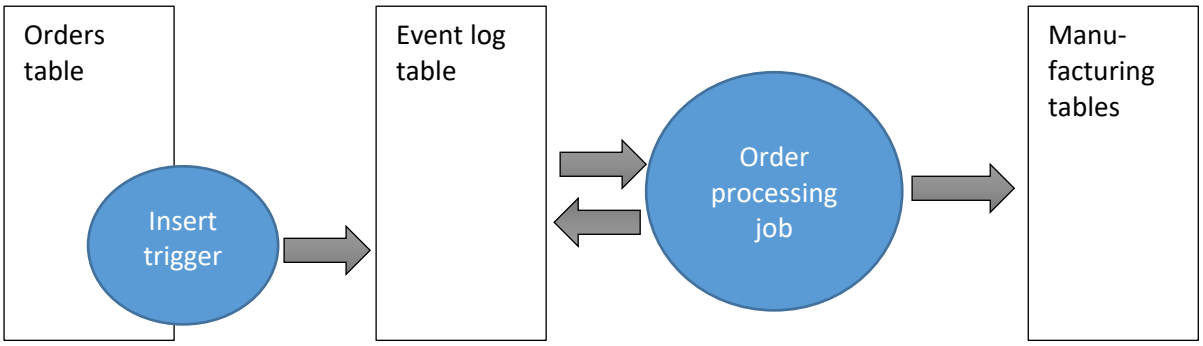
Since the trading system is a 'black box', we must rely on database level events. Every time an order is created or modified, we must run the required (rather complex) logic on the database that creates or changes the required records in the SL tables like Products. Thus both the writing and the processing of emails will be unnecessary.

It is, however, vital that our solution *should not at the least interfere* with the trading system. It cannot significantly slow down the order saving process, nor may any error that may occur while processing an order event on the SL side be propagated back to the trading system.

For this reason, we use the *loose coupling* concept. We only log the INSERT and UPDATE events on orders via a trigger in a special table, and process these events in batches executed by a scheduled job. The job also keeps track of the state and results of the processing of each event. Since the processing of the event is performed out-of-process, a processing error does not manifest as an error in the trading system.

*Note: a **trigger** is a special stored procedure that is invoked automatically by the database management system upon database events like table INSERT, UPDATE or DELETE. For more details, see the Appendix of this chapter.*

System overview:



Tight coupling

In the example below we create a new insert trigger on the Orders table that runs long and throws an exception, thus disabling the order saving process.

```
drop trigger tr_demo_bad
go
create trigger tr_demo_bad on orders for insert as
declare @orderid int
select @orderid=OrderID from inserted
print 'New order ID: ' + cast(@orderid as varchar(50))
waitfor delay '00:00:10' --10 s
select 1/0 --we make an error
go
--test #1:  with both last lines commented out
insert  Orders (CustomerID, OrderDate) values ('AROUT', GETDATE())
--restore table
delete Orders where CustomerID='AROUT' and EmployeeID is null
--test #2: recreate the trigger, with the last lines commented out
insert  Orders (CustomerID, OrderDate) values ('AROUT', GETDATE())
--we have long to wait, but there is no error
--restore table
delete Orders where CustomerID='AROUT' and EmployeeID is null
--test #3: recreate the trigger, with all lines
insert  Orders (CustomerID, OrderDate) values ('AROUT', GETDATE())
--we have long to wait, then we have the message:
'New order ID: 11094
Msg 8134, Level 16, State 1, Procedure tr_demo_bad, Line 6 [Batch Start Line 276]
Divide by zero error encountered.
The statement has been terminated.'
select * from Orders where CustomerID='AROUT' and EmployeeID is null
--no such record, because
--the insert statemant has been rolled back -> we crashed the trading system
```

This is exactly what we do NOT want. We implement *loose coupling* instead of *tight coupling*.

The loosely coupled system

The idea is that the trigger only saves the events into a log table. We then process the table with a stored procedure.

The log table and the trigger

The trigger uses the virtual tables *inserted* and *deleted*. This trigger can process multi-record INSERTs and UPDATEs.

```
--the log table
go
--drop table order_log
go
create table order_log (
       event_id int IDENTITY (1, 1) primary key ,
       event_type varchar(50) NOT NULL ,
       order_id int NOT NULL ,
       orderitem_id int NULL ,
       status int NOT NULL default(0),
       time_created datetime NOT NULL default(getdate()) ,
       time_process_begin datetime NULL ,
       time_process_end datetime NULL ,
       process_duration as datediff(second, time_process_begin, time_process_end)
)
go
drop trigger tr_log_order
go
```

```
create trigger tr_log_order ON Orders for insert, update as
declare @orderid int
select @orderid=orderid from inserted --there can be more then a single record in inserted
print 'OrderID of the LAST record: ' + cast(@orderid as varchar(50))
if update(orderid) begin --if the orderid has changed, then this is an INSERT
	print 'Warning: new order'
	insert order_log (event_type, order_id)  --status, time_created use default
		select 'new order', orderid from inserted
end else if update(shipaddress) or update(shipcity) begin --shipaddress or shipcity has
changed
	print 'Warning: address changed'
	insert order_log (event_type, order_id)
		select 'address changed', orderid from inserted
end else begin  --other change
	print 'Warning: other change'
	insert order_log (event_type, order_id)
		select 'other change', orderid from inserted
end
go

--test #1
insert  Orders (CustomerID, OrderDate) values ('AROUT', GETDATE())
select * from order_log
--we have one new record in the log table

--test #2
insert  Orders (CustomerID, OrderDate) values ('AROUT', GETDATE()), ('HANAR', GETDATE())
select * from order_log
--we have two new records in the log table

--test #3
update Orders set ShipVia = 3 where OrderID in (11097, 11096) --these are the IDs of test
#2
select * from order_log
--we have two new records of the type 'other change'

--restore the tables
delete Orders where CustomerID in ('AROUT', 'HANAR') and EmployeeID is null
delete order_log
```

The stored procedure for processing new orders

We expect that the items of a new order are inserted subsequently after the order record is created.

```
--a simple stored procedure that processes a new order
--and returns 0 if all of its items could be committed to the inventory without error
--demonstrating also the use of output parameters
drop proc  sp_commit_new_order_to_inventory
go
create procedure sp_commit_new_order_to_inventory
@orderid int,
@result int output
as
begin try
	update products set unitsInStock = unitsInStock - od.quantity
	from products p inner join [Order Details] od on od.ProductID=p.ProductID
	where od.OrderID=@orderid
	set @result=0
end try
begin catch
	print '  Inventory error: '+ ERROR_MESSAGE() + ' (' + cast(ERROR_NUMBER() as
varchar(20)) + ')'
	set @result=1
end catch
go
```

```
--test
select * from order_log --11097
select * from Products where ProductID=10 --unitsinstock =31
select * from Products where ProductID=9 --unitsinstock =29
insert [Order Details]  (orderid, productid, quantity, UnitPrice, Discount)
values (11097, 9, 10, 30, 0),(11097, 10, 40, 30, 0)  --the second item will cause an error
in sp_commit_new_order_to_inventory
go
declare @res int
exec sp_commit_new_order_to_inventory 11097, @res output
print @res
exec sp_commit_new_order_to_inventory 11096, @res output
print @res
go
--check: no change in unitsinstock (OK)
select * from Products where ProductID=10 --unitsinstock =31
select * from Products where ProductID=9 --unitsinstock =29
```

The stored procedure for processing the event log

Since completely different actions are to be taken depending on the event type, we use a cursor to iterate the order log.

```
--stored procedure for processing the order_log
--drop proc sp_order_process
go
create proc sp_order_process as
declare @event_id int, @event_type varchar(50), @order_id int, @result int
declare cursor_events cursor forward_only static
       for
       select  event_id, event_type, order_id
       from order_log where status=0 --we only care for the unprocessed events

set xact_abort on
set nocount on
open cursor_events
fetch next from cursor_events into @event_id, @event_type, @order_id
while @@fetch_status = 0
begin
       print 'Processing event ID=' + cast(@event_id as varchar(10)) + ', Order ID=' +
cast(@order_id as varchar(10))
       update order_log set time_process_begin=getdate() where event_id=@event_id
       begin tran
       set @result = null
       if @event_type = 'new order' begin
              print '  Processing new order...'
              exec sp_commit_new_order_to_inventory @order_id, @result output
       end else if @event_type = 'address changed' begin
              print '  Processing address changed...'
              waitfor delay '00:00:01' --we only simulate the processing of other event
types
              set @result=0
       end else if @event_type = 'other change' begin
              print '  Processing other change...'
              waitfor delay '00:00:01'
              set @result=0
       end else begin
              print '  Unknown event type...'
              waitfor delay '00:00:01'
              set @result=1
       end

       if @result=0 begin
              print 'Event processing OK'
              commit tran
```

```
        end else begin
                print 'Event processing failed'
                rollback tran
        end
        print ''
        update order_log set time_process_end=getdate(),
                status=case when @result=0 then 2 else 1 end
                where event_id=@event_id
        fetch next from cursor_events into @event_id, @event_type, @order_id
end
close cursor_events deallocate cursor_events
go

--teszt
update order_log set status=0
select *from orders where EmployeeID is null
select * from order_log
exec dbo.sp_order_process
select * from order_log

--we get:
Processing event ID=5, Order ID=11097
  Processing new order...
  Inventory error: The UPDATE statement conflicted with the CHECK constraint etc.
Event processing failed

Processing event ID=6, Order ID=11096
  Processing new order...
Event processing OK
```

The scheduled job that calls the event log processor

We implement the job using the SSMS GUI and check its operation in the Job Activity Monitor. Follow the steps below.

1. Open the Configuration Manager and start the SQL Server Agent service
2. In the SSMS, open the Jobs under SQL Server Agent, and select New job
3. Specify the name of the job as order_job, Northwind as the target database
4. Add a new job step with the T-SQL statement ‘exec dbo.sp_order_process’
5. Specify a schedule that runs the job once a minute
6. Check that the new job appears in the Jobs list (refresh), and check the history of the job
7. Add a new order and verify that the event is processed by the job within 1 minute i.e. the event has been propagated in the LS division as planned.
8. Finally, delete the job and the trigger

APPENDIX: A short overview on triggers

Triggers are special procedures stored on the server and run automatically when a pre-defined condition is satisfied. SQL Server supports the following types of triggers with respect to the trigger event:

- DML triggers (table level triggers) that are executed when a DELETE, INSERT or UPDATE action is performed on a table
- DDL triggers (database level triggers) that are fired when the schema of the database changes e.g. a table is created
- Logon triggers (server level triggers) that are fired after the authentication phase of logging in finishes

We focus now on DML triggers. The definition of the trigger includes the target table, the trigger event (DELETE, INSERT or UPDATE) and the mode of operation. SQL server supports the following operational modes:

- AFTER: fired after the successful execution of the specified SQL statement. This means that all eventual check and other constraints and cascade updates/deletes associated with the DML statement have executed successfully. We can place multiple triggers on the same object, even of the same type, like two INSERT triggers. In this case, the order of execution can be influenced by setting trigger properties.
- INSTEAD OF: the DML statement is not executed at all, only the trigger.

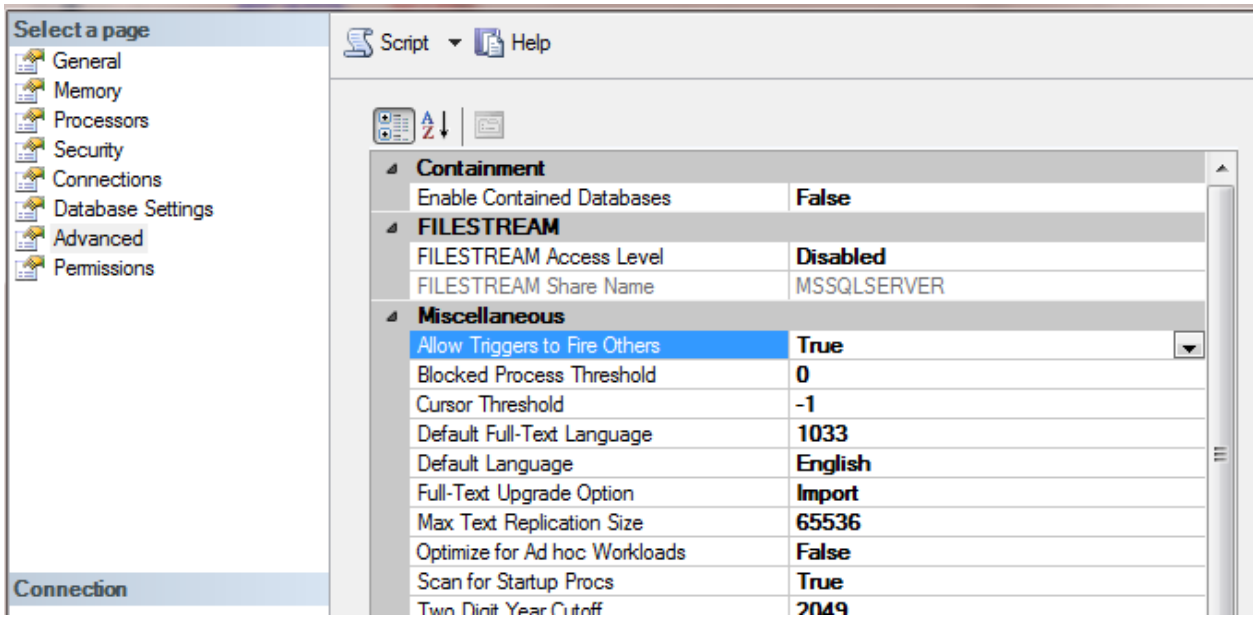
The records modified by the DML statement can be accessed by the trigger code via special logical tables. SQL server provides the following two logical tables:

- **'deleted'**: holds the records deleted form the table in case of a DELETE trigger or the *original* (old) records updated in case of an UPDATE trigger. An update is logically equivalent to a delete followed by an insert. The deleted table is empty in case of an INSERT trigger.
- **'inserted'**: holds the records inserted by an INSERT statement or the *new* records updated by an UPDATE trigger. The inserted table is empty in case of a DELETE trigger.

SQL server also supports the **update**([field name]) function available in INSERT or UPDATE triggers that returns true if the DML statement changed the specified field. The field cannot be a computed column.

If the trigger raises an error, the DML statement is rolled back.

A trigger may run code that invokes other triggers or even the same trigger in a recursive manner, up to 32 levels on MS SQL server. This feature is controlled via the nested triggers server option (see below).



Cases when the use of a DML trigger is recommended

- Administration functions such as maintaining a log or keeping old values of changed records in backup tables.
- Enforcing data integrity rules that follow from the business logic and that are beyond the scope of simple primary key, foreign key or check constraints. Example in the Northwind database:
    - We do not send heavy packages overseas. Therefore we refuse orders with a freight over 200 kg that has a ShipCountry not equal to USA. This can be implemented by an INSERT AFTER or INSERT INSTEAD OF trigger on the Orders table. Such checks should of course be built into the client software, however, database level integrity enforcement can prevent application errors or hacking.
- Automating business workflow processes. Examples in the Northwind database:
    - We send an automated email to the customer when the shipping date is decided i.e. when the ShippedDate field of an order is set (UPDATE trigger)
    - We automatically send an order to our wholesale supplier when the UnitsInStock of a Product drops below the ReorderLevel (UPDATE or INSERT trigger)
    - We automatically update the UnitsInStock field of the Products table when the quantity field in a corresponding Order Detail record changes (UPDATE trigger)

WARNING: the operation of triggers is 'silent', and severe problems may result from forgetting about them. For example, if the administrator restores the Order Items tables from a backup copy with an UPDATE statement without first disabling the trigger…

For more examples on SQL Server triggers see:

- http://sqlhints.com/2016/02/28/inserted-and-deleted-logical-tables-in-sql-server/

PRACTICE: write an UPDATE trigger for the Order Details table. When the quantity changes, update the UnitsInStock of the product. You can assume that only one Order Details record is updated at a time.

PRACTICE: Assume that in the problem above more than Order Details records are updated at a time. Refine the code accordingly.